FEA Beam Solver with Transient Force Support

Nishit Singh

Design Oriented Project under Dr. Gaurav Sharma, May 2026

Abstract

This paper reports on the transient finite element analysis of a beam. As input, the solver takes in beam properties, time step parameters, and forces as functions of time. As the output, the solver gives the user access to `.csv` files and a `.gif` file for easy visualization.

1 Defining the Input Setup

1.1 Beam Properties

To edit beam properties, navigate to the `make_beam()` function in the file `main.cpp`. The solver takes as input properties length in m (`cfg.L`), number of elements (`cfg.n_elem`), Young's modulus in Pascals (`cfg.E`), second moment of inertia in m^4 (`cfg.I`), cross sectional area in m^2 (`cfg.a`), density in $\frac{kg}{m^3}$ (`cfg.rho`), damping ration (`cfg.zeta`), and boundary conditions (fixed, free, and pinned) (`cfg.bc_left` and `cfg.bc_right`).

1.2 Force Definitions

1.2.1 Distributed Loads

To edit force definitions, navigate to the `make_forces()` function in the file `main.cpp`. The solver takes as input forces in the axial (`fc.qx`) and transverse (`fc.qy`) directions. Every expression valid in `CPP` is valid as an expression here. For example, for force F,

$$F = F(x,t) = 500sin(\frac{x}{L}t),$$

we input an expression like such : `return 500.0 * std::sin((x / L)*t);`

1.2.2 Point Loads

For point loads, the solver admits forces at a point in meters (can be a function of time), with a certain magnitude in N, and a application angle θ in degrees. To edit these parameters, in the module `make_forces()`, add the following code block.

```
pf.x_fn     = [&cfg](double t) { return cfg.L; };          // position along beam [m]
pf.mag_fn   = [](double t)     { return 5000.0; };         // magnitude [N]
pf.angle_fn = [](double t)     { return 90.0; };           // angle from axial axis [degrees]
fc.point_forces.push_back(pf);
PointForce pf;
```

1.3 Timestep Definition

The solver works on a Newman-β solver, which is unconditionally stable under time quantization. To edit time-step information, navigate to to the `make_solver_config()` function in `main.cpp` and edit the following code block to represent correct time range, and time steps.

```
SolverConfig make_solver_config() {
SolverConfig sc;
sc.t_start = 0.0;   // start time [s]
sc.t_end   = 2.0;   // end time [s]
sc.n_steps = 500;   // number of time steps
}
```

2 The Solver

Each node carries with itself 3 degrees of freedom, namely axial and transverse displacement, and the rotation of the cross-section (the slope). Therefore, the universal DOF vector is -

$$U = [u_i, v_i, \theta_i]^T; \forall i$$

where u_i is the axial displacement of the ith node, v_i is the transverse displacement, and θ_i is the rotation. The mapping is $3i$ for u_i, $(3i+1)$ for v_i, and $(3i+2)$ for θ_i.

2.1 Governing Equations

The Euler-Bernoulli equation couples two independent equations in its strong form. The equations for the axial and transverse displacement are -

$$\rho A \frac{d^2U}{dt^2} - EA\frac{d^2U}{dx^2} = q_x(x,t)$$

$$\rho A \frac{d^2V}{dt^2} - EA\frac{d^4V}{dx^2} = q_y(x,t)$$

Baked into these equations are the usual small strain assumptions, homogeneous and uniform material, and no shear deformation. Deriving the weak form of these equations, we multiply by test functions and integrate by parts. The explicit derivation is doe in the appendix. The governing equations can be written in the matrix form -

$$\boxed{\mathbf{M}\,\ddot{\mathbf{U}}(t) + \mathbf{C}\,\dot{\mathbf{U}}(t) + \mathbf{K}\,\mathbf{U}(t) = \mathbf{F}(t)}$$

Where $\mathbf{M}$ is the global mass matrix, $\mathbf{C}$ is the damping matrix, $\mathbf{K}$ is stiffness matrix, $\mathbf{U}(t)$ is displacement vector, and $\mathbf{F}(t)$ is the time-varying force.

2.2 Element formulation

For axial displacement, linear Lagrange shape functions are utilized to interpolate the axial field $u(x)$ over the length.

$$u(\xi) = N_1(\xi)u_1 + N_2(\xi)u_2$$

$$N_1(\xi) = 1 - \xi$$

$$N_2 = \xi$$

$$\xi \in [0, 1]$$

For transverse displacement, Hermite cubic shape polynomials are utilized. The interpolation is as follows -

$$v(\xi) = H_1(\xi)v_1 + H_2(\xi)\theta_1 + H_3(\xi)v_2 + H_4(\xi)\theta_2$$

$$H_1(\xi) = 1 - 3\xi^2 + 2\xi^3$$

$$H_2(\xi) = L_e(\xi - 2\xi^2 + \xi^3)$$

$$H_3(\xi) = 3\xi^2 - 2\xi^3$$

$$H_4(\xi) = L_e(\xi^3 - \xi^2)$$

For the element stiffness matrix, we solve the shape function interpolations on the weak form integrals analytically. Done in the appendix, the final results are -

Axial stiffness (2 X 2 block) -

$$\frac{EA}{L_e}\begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix}$$

Bending stiffness (4 X 4 block) -

$$\mathbf{K}_{bend} = \frac{EI}{L_e^3}\begin{bmatrix} 12 & 6L_e & -12 & 6L_e \\ 6L_e & 4L_e^2 & -6L_e & 2L_e^2 \\ -12 & -6L_e & 12 & -6L_e \\ 6L_e & 2L_e^2 & -6L_e & 4L_e^2 \end{bmatrix}$$

and finally, the combined 6 X 6 block -

$$\mathbf{K}_{bend} = \frac{EI}{L_e^3}\begin{bmatrix} 12 & 6L_e & -12 & 6L_e \\ 6L_e & 4L_e^2 & -6L_e & 2L_e^2 \\ -12 & -6L_e & 12 & -6L_e \\ 6L_e & 2L_e^2 & -6L_e & 4L_e^2 \end{bmatrix}$$

Now, the consistent mass matrix is derived from the kinetic energy -

$$T = \frac{1}{2}\int_0^L \rho A(\dot{u}^2 + \dot{v}^2)dx$$

With calculations in the appendix, the final expression for the mass matrix is -

$$\mathbf{M}_e = \begin{bmatrix} \frac{\rho A L_e}{3} & 0 & 0 & \frac{\rho A L_e}{6} & 0 & 0 \\ 0 & \frac{156\rho A L_e}{420} & \frac{22\rho A L_e^2}{420} & 0 & \frac{54\rho A L_e}{420} & -\frac{13\rho A L_e^2}{420} \\ 0 & \frac{22\rho A L_e^2}{420} & \frac{4\rho A L_e^3}{420} & 0 & \frac{13\rho A L_e^2}{420} & -\frac{3\rho A L_e^3}{420} \\ \frac{\rho A L_e}{6} & 0 & 0 & \frac{\rho A L_e}{3} & 0 & 0 \\ 0 & \frac{54\rho A L_e}{420} & \frac{13\rho A L_e^2}{420} & 0 & \frac{156\rho A L_e}{420} & -\frac{22\rho A L_e^2}{420} \\ 0 & -\frac{13\rho A L_e^2}{420} & -\frac{3\rho A L_e^3}{420} & 0 & -\frac{22\rho A L_e^2}{420} & \frac{4\rho A L_e^3}{420} \end{bmatrix}$$

2.3 Numerical Integration

The standard Gauss quadrature rules are defined on the interval [-1,1]. Here, a parametric coordinate $\xi \in [0, 1]$ is used, where $\xi = 0$ corresponds to node 1 and $\xi = 1$ to node 2. The mapping to the physical coordinate is -

$$x = x_1 + \xi L_e, \qquad dx = L_e\, d\xi$$

The Jacobian of this transformation is $J = L_e$. The [0,1] Gauss points and weights are obtained from the standard [-1,1] rule via the linear map -

$$\xi = \frac{\eta + 1}{2}$$

which introduces a scaling factor of 1/2 in the weights. The tabulated weights used in the implementation are already transformed to the interval [0,1], and therefore include this factor. The final integration weight at each Gauss point is -

$$w_g = w_g^{[0,1]} \cdot L_e$$

In the code -

```
double xg = x1 + xi * Le;
double w  = gw[g] * Le;
```

A 4-point Gauss rule integrates polynomials of degree up to 7 exactly. Since the Hermite shape functions are cubic, the integrand $H_i(x)q(x,t)$ is polynomial only if q is polynomial. For smooth $q(x,t)$, Gauss quadrature converges rapidly, and a 4-point rule provides high accuracy in practice.

2.4 Time Integration

At each time step, the semi-discrete equations of motion are-

$$\mathbf{M}\,\mathbf{a}(t) + \mathbf{C}\,\mathbf{v}(t) + \mathbf{K}\,\mathbf{u}(t) = \mathbf{F}(t)$$

$$\mathbf{a} = \ddot{\mathbf{u}}, \quad \mathbf{v} = \dot{\mathbf{u}}, \quad \mathbf{u} = \mathbf{U}$$

This is a second-order ordinary differential equation in time. The Newmark-β method converts it into a sequence of linear algebraic problems, one per time step, each involving only the displacement at the new time t_{n+1}. The method assumes the following update formulas-

$$\mathbf{u}_{n+1} = \mathbf{u}_n + \Delta t\,\mathbf{v}_n + \Delta t^2\left[(0.5-\beta)\mathbf{a}_n + \beta\mathbf{a}_{n+1}\right]$$

$$\mathbf{v}_{n+1} = \mathbf{v}_n + \Delta t\left[(1-\gamma)\mathbf{a}_n + \gamma\mathbf{a}_{n+1}\right]$$

Define the predictor-

$$\mathbf{u}_{\text{pred}} = \mathbf{u}_n + \Delta t\,\mathbf{v}_n + \Delta t^2(0.5-\beta)\mathbf{a}_n;$$

$$\mathbf{a}_{n+1} = \frac{\mathbf{u}_{n+1} - \mathbf{u}_{\text{pred}}}{\beta\Delta t^2}$$

Substituting into the equation of motion at t_{n+1} -

$$\mathbf{M}\,\mathbf{a}_{n+1} + \mathbf{C}\,\mathbf{v}_{n+1} + \mathbf{K}\,\mathbf{u}_{n+1} = \mathbf{F}_{n+1}$$

leads to the effective system -

$$\mathbf{K}_{\text{eff}}\,\mathbf{u}_{n+1} = \mathbf{F}_{\text{eff}}$$

where -

$$\mathbf{K}_{\text{eff}} = \mathbf{K} + \frac{1}{\beta\Delta t^2}\mathbf{M} + \frac{\gamma}{\beta\Delta t}\mathbf{C}$$

The matrix $\mathbf{K}_{\text{eff}}$ is constant for linear systems and can be factorised once before the time-stepping loop. Only the right-hand side $\mathbf{F}_{\text{eff}}$ changes at each step. The choice of $\beta = 0.25, \gamma = 0.5$ correspond to constant-average-acceleration method (also called the trapezoidal method). This choice is unconditionally stable for linear systems and accurate upto the second order in time.

2.5 Rayleigh Damping

In physical systems, damping arises from multiple mechanisms such as material hysteresis, joint friction, and acoustic radiation. In finite element models, these effects are commonly approximated using Rayleigh damping, in which the damping matrix is expressed as a linear combination of the mass and stiffness matrices:

$$\mathbf{C} = \alpha_M\mathbf{M} + \alpha_K\mathbf{K}$$

This form of damping is classical, meaning that it preserves the modal basis of the undamped system. As a result, the equations of motion can be decoupled in modal coordinates, which is a mathematical convenience. The damping ratio ζ_i for mode i, with natural frequency ω_i, is given by -

$$\zeta_i = \frac{\alpha_M}{2\omega_i} + \frac{\alpha_K \omega_i}{2}$$

The full Rayleigh model requires two target frequencies to determine both α_M and α_K. In this implementation, only the mass-proportional term is retained -

$$\mathbf{C} = \alpha_M \mathbf{M}$$

The stiffness-proportional term is omitted because it requires knowledge of at least two natural frequencies to calibrate both coefficients, it introduces frequency-dependent damping that can overdamp higher modes and for lightly damped systems dominated by the first mode, mass-proportional damping provides a reasonable approximation. The coefficient α_M is chosen to match a target damping ratio ζ for the first mode -

$$\alpha_M = 2\zeta\omega_1$$

In the implementation, ω_1 is approximated using a diagonal estimate -

```
double omega1 = sqrt(K[1][1] / M[1][1]);
double alpha_M = 2.0 * zeta * omega1;
```

This provides a rough estimate of the first natural frequency, avoiding the need for a full eigenvalue solve. Incorporating into time integration with $\mathbf{C} = \alpha_M \mathbf{M}$, the effective stiffness matrix in the Newmark scheme becomes -

$$\mathbf{K}_{\text{eff}} = \mathbf{K} + \left(\frac{1}{\beta \Delta t^2} + \frac{\gamma \alpha_M}{\beta \Delta t} \right) \mathbf{M}$$

The damping contribution also appears in the right-hand side through the velocity predictor -

$$\mathbf{C}\,\mathbf{v}_{\text{pred}} = \alpha_M \mathbf{M} \mathbf{v}_{\text{pred}}$$

2.6 Linear System Solver

At each time step, the solver must solve the linear system:

$$\mathbf{K}_{\text{eff}}\,\Delta\mathbf{u} = \mathbf{RHS}$$

This system is treated as a dense $n \times n$ problem and solved using LU decomposition with partial pivoting, implemented in `lu_solve()` in `src/linalg.cpp`.
The LU decomposition factors the matrix as -

$$\mathbf{K}_{\text{eff}} = \mathbf{PLU}$$

where $\mathbf{P}$ is a permutation matrix (row swaps), $\mathbf{L}$ is unit lower-triangular, $\mathbf{U}$ is upper-triangular. Once factorized, the solution is obtained via two triangular solves:

$$\mathbf{Ly} = \mathbf{PRHS}, \qquad \mathbf{U}\Delta\mathbf{u} = \mathbf{y}$$

The factorisation costs $O(n^3)$, while each solve costs $O(n^2)$. Without pivoting, the elimination process divides by the diagonal entry A_{kk} at each step. If this value is zero or very small, the division either fails or produces large intermediate values that amplify round-off errors. Even when the matrix is theoretically non-singular, floating-point arithmetic can produce small pivots during elimination. Partial pivoting mitigates this by selecting the largest available pivot in the current column:

```
for (int i = k+1; i < n; i++)
    if (abs(A[i][k]) > maxval) { maxval = abs(A[i][k]); maxrow = i; }

swap(A[k], A[maxrow]);
swap(b[k], b[maxrow]);
```

This strategy improves numerical stability by controlling the growth of intermediate values during factorisation. Without pivoting, the solver can become numerically unstable and produce inaccurate results, particularly for ill-conditioned systems (e.g. meshes with widely varying element sizes or large stiffness contrasts).

The LU factorisation is performed once per simulation, since $\mathbf{K}_{\text{eff}}$ is constant for linear problems. This costs $O(n^3)$. At each time step, the forward and backward substitutions cost $O(n^2)$. For a beam discretisation with $n = 3(n_{\text{elem}} + 1)$ degrees of freedom, typical problem sizes ($n \approx 60$–90) are computationally inexpensive. For larger systems ($n > 500$), more efficient methods such as banded or sparse solvers would significantly reduce computational cost.

APPENDIX

Deriving the matrix form of the Euler Bernoulli Equations

Axial equation:

$$\rho A\ddot{u} - EA\,u'' = q_x(x,t)$$

Multiply by test function w and integrate over $[0, L]$:

$$\int_0^L w\left(\rho A\ddot{u} - EA\,u''\right)\,dx = \int_0^L w\,q_x\,dx$$

Integrate by parts:

$$\int_0^L w(-EA\,u'')\,dx = \int_0^L EA\,w'u'\,dx - \left[EA\,wu'\right]_0^L$$

Weak form:

$$\int_0^L \rho A\,w\,\ddot{u}\,dx + \int_0^L EA\,w'u'\,dx = \int_0^L w\,q_x\,dx + \left[EA\,wu'\right]_0^L$$

Transverse (bending) equation:

$$\rho A\ddot{v} + EI\,v'''' = q_y(x,t)$$

Multiply by test function z and integrate:

$$\int_0^L z\left(\rho A\ddot{v} + EI\,v''''\right)\,dx = \int_0^L z\,q_y\,dx$$

First integration by parts:

$$\int_0^L z\,EI\,v''''\,dx = \left[EI\,zv'''\right]_0^L - \int_0^L EI\,z'v'''\,dx$$

Second integration by parts:

$$\int_0^L EI\,z'v'''\,dx = \left[EI\,z'v''\right]_0^L - \int_0^L EI\,z''v''\,dx$$

Weak form:

$$\int_0^L \rho A\,z\,\ddot{v}\,dx + \int_0^L EI\,z''v''\,dx = \int_0^L z\,q_y\,dx + \left[EI\,zv'''\right]_0^L - \left[EI\,z'v''\right]_0^L$$

Deriving the stiffness matrix

Element DOF vector:

$$\mathbf{d}_e = \begin{bmatrix} u_1 & v_1 & \theta_1 & u_2 & v_2 & \theta_2 \end{bmatrix}^T$$

Axial field interpolation:

$$u(x) = N_1(x)u_1 + N_2(x)u_2$$

$$N_1 = 1 - \frac{x}{L_e}, \quad N_2 = \frac{x}{L_e}$$

$$\frac{dN_1}{dx} = -\frac{1}{L_e}, \quad \frac{dN_2}{dx} = \frac{1}{L_e}$$

Axial strain:

$$\varepsilon = \frac{du}{dx} = N_1' u_1 + N_2' u_2$$

Axial stiffness:

$$K_{ij}^{(axial)} = \int_0^{L_e} EA\, N_i' N_j'\, dx$$

$$K_{11} = EA \int_0^{L_e} \left(-\frac{1}{L_e}\right)^2 dx = \frac{EA}{L_e}$$

$$K_{12} = EA \int_0^{L_e} \left(-\frac{1}{L_e}\right)\left(\frac{1}{L_e}\right) dx = -\frac{EA}{L_e}$$

$$K_{21} = -\frac{EA}{L_e}, \quad K_{22} = \frac{EA}{L_e}$$

$$\mathbf{K}_{axial} = \frac{EA}{L_e} \begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix}$$

Bending field interpolation:

Let $\xi = x/L_e$

$$v(x) = H_1(\xi)v_1 + H_2(\xi)\theta_1 + H_3(\xi)v_2 + H_4(\xi)\theta_2$$

$$H_1 = 1 - 3\xi^2 + 2\xi^3, \quad H_2 = L_e(\xi - 2\xi^2 + \xi^3)$$

$$H_3 = 3\xi^2 - 2\xi^3, \quad H_4 = L_e(\xi^3 - \xi^2)$$

Second derivatives:

$$\frac{d^2}{dx^2} = \frac{1}{L_e^2}\frac{d^2}{d\xi^2}$$

$$H_1'' = \frac{-6+12\xi}{L_e^2}, \quad H_2'' = \frac{-4+6\xi}{L_e}$$

$$H_3'' = \frac{6-12\xi}{L_e^2}, \quad H_4'' = \frac{-2+6\xi}{L_e}$$

Bending stiffness:

$$K_{ij}^{(bend)} = \int_0^{L_e} EI\, H_i'' H_j''\, dx$$

$$= \int_0^1 EI\, H_i''(\xi) H_j''(\xi)\, L_e\, d\xi$$

Example terms:

$$K_{11} = EI \int_0^1 \left(\frac{-6+12\xi}{L_e^2}\right)^2 L_e\, d\xi = \frac{EI}{L_e^3}\int_0^1 (36 - 144\xi + 144\xi^2)\, d\xi = \frac{12EI}{L_e^3}$$

$$K_{12} = EI \int_0^1 \left(\frac{-6+12\xi}{L_e^2}\right)\left(\frac{-4+6\xi}{L_e}\right) L_e\, d\xi = \frac{6EI}{L_e^2}$$

Full bending matrix:

$$\mathbf{K}_{bend} = \frac{EI}{L_e^3}\begin{bmatrix} 12 & 6L_e & -12 & 6L_e \\ 6L_e & 4L_e^2 & -6L_e & 2L_e^2 \\ -12 & -6L_e & 12 & -6L_e \\ 6L_e & 2L_e^2 & -6L_e & 4L_e^2 \end{bmatrix}$$

Full element stiffness matrix (6×6):

$$\mathbf{K}_e = \begin{bmatrix} \frac{EA}{L_e} & 0 & 0 & -\frac{EA}{L_e} & 0 & 0 \\ 0 & \frac{12EI}{L_e^3} & \frac{6EI}{L_e^2} & 0 & -\frac{12EI}{L_e^3} & \frac{6EI}{L_e^2} \\ 0 & \frac{6EI}{L_e^2} & \frac{4EI}{L_e} & 0 & -\frac{6EI}{L_e^2} & \frac{2EI}{L_e} \\ -\frac{EA}{L_e} & 0 & 0 & \frac{EA}{L_e} & 0 & 0 \\ 0 & -\frac{12EI}{L_e^3} & -\frac{6EI}{L_e^2} & 0 & \frac{12EI}{L_e^3} & -\frac{6EI}{L_e^2} \\ 0 & \frac{6EI}{L_e^2} & \frac{2EI}{L_e} & 0 & -\frac{6EI}{L_e^2} & \frac{4EI}{L_e} \end{bmatrix}$$

Deriving the Mass Matrix

Element DOF vector:

$$\mathbf{d}_e = \begin{bmatrix} u_1 & v_1 & \theta_1 & u_2 & v_2 & \theta_2 \end{bmatrix}^T$$

Kinetic energy:

$$T = \frac{1}{2}\int_0^{L_e} \rho A \left(\dot{u}^2 + \dot{v}^2\right) dx$$

$$= \frac{1}{2}\dot{\mathbf{d}}_e^T \mathbf{M}_e \dot{\mathbf{d}}_e$$

Axial interpolation:

$$u(x) = N_1 u_1 + N_2 u_2, \quad N_1 = 1 - \frac{x}{L_e}, \quad N_2 = \frac{x}{L_e}$$

$$\dot{u}(x) = N_1 \dot{u}_1 + N_2 \dot{u}_2$$

Axial mass matrix:

$$\mathbf{M}_{axial} = \int_0^{L_e} \rho A \begin{bmatrix} N_1 \\ N_2 \end{bmatrix} \begin{bmatrix} N_1 & N_2 \end{bmatrix} dx$$

$$M_{11} = \rho A \int_0^{L_e} \left(1 - \frac{x}{L_e}\right)^2 dx = \rho A \left[L_e - L_e + \frac{L_e}{3}\right] = \frac{\rho A L_e}{3}$$

$$M_{12} = \rho A \int_0^{L_e} \left(1 - \frac{x}{L_e}\right)\left(\frac{x}{L_e}\right) dx = \frac{\rho A L_e}{6}$$

$$\mathbf{M}_{axial} = \frac{\rho A L_e}{6} \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$$

Bending interpolation:

Let $\xi = x/L_e$, $dx = L_e d\xi$

$$v(x) = H_1 v_1 + H_2 \theta_1 + H_3 v_2 + H_4 \theta_2$$

$$\dot{v}(x) = H_1 \dot{v}_1 + H_2 \dot{\theta}_1 + H_3 \dot{v}_2 + H_4 \dot{\theta}_2$$

Bending mass matrix:

$$\mathbf{M}_{bend} = \int_0^{L_e} \rho A \begin{bmatrix} H_1 \\ H_2 \\ H_3 \\ H_4 \end{bmatrix} \begin{bmatrix} H_1 & H_2 & H_3 & H_4 \end{bmatrix} dx$$

$$= \rho A L_e \int_0^1 \mathbf{H}^T(\xi)\mathbf{H}(\xi)\, d\xi$$

Example entries:

$$M_{11} = \rho A L_e \int_0^1 (1 - 3\xi^2 + 2\xi^3)^2 d\xi = \rho A L_e \cdot \frac{156}{420}$$

$$M_{12} = \rho A L_e \int_0^1 H_1(\xi) H_2(\xi) d\xi = \rho A L_e \cdot \frac{22 L_e}{420}$$

Full bending mass matrix:

$$\mathbf{M}_{bend} = \frac{\rho A L_e}{420} \begin{bmatrix} 156 & 22L_e & 54 & -13L_e \\ 22L_e & 4L_e^2 & 13L_e & -3L_e^2 \\ 54 & 13L_e & 156 & -22L_e \\ -13L_e & -3L_e^2 & -22L_e & 4L_e^2 \end{bmatrix}$$

Full element mass matrix (6×6):

$$\mathbf{M}_e = \begin{bmatrix} \frac{\rho A L_e}{3} & 0 & 0 & \frac{\rho A L_e}{6} & 0 & 0 \\ 0 & \frac{156\rho A L_e}{420} & \frac{22\rho A L_e^2}{420} & 0 & \frac{54\rho A L_e}{420} & -\frac{13\rho A L_e^2}{420} \\ 0 & \frac{22\rho A L_e^2}{420} & \frac{4\rho A L_e^3}{420} & 0 & \frac{13\rho A L_e^2}{420} & -\frac{3\rho A L_e^3}{420} \\ \frac{\rho A L_e}{6} & 0 & 0 & \frac{\rho A L_e}{3} & 0 & 0 \\ 0 & \frac{54\rho A L_e}{420} & \frac{13\rho A L_e^2}{420} & 0 & \frac{156\rho A L_e}{420} & -\frac{22\rho A L_e^2}{420} \\ 0 & -\frac{13\rho A L_e^2}{420} & -\frac{3\rho A L_e^3}{420} & 0 & -\frac{22\rho A L_e^2}{420} & \frac{4\rho A L_e^3}{420} \end{bmatrix}$$